Subiecte Teste 2025

Student 01

1. Implementați o funcție `Whiskey41()` care primește 2 numere `double` și le inversează dacă ambele sunt pare. **(2p)**
2. Implementați o funcție `November23()` care primește o listă de `short` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `short` și returnează un `boolean`, iar a doua funcție primește un `short` și returnează un `short`.

`November23()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Motocicleta`, care conține: **(1p)**
  - proprietățile: `marca` `model` `viteza_maxima` (public get, protected set)
  - un constructor pentru inițializarea proprietăților
  - o metodă abstractă `void Salut()`
  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor
- O clasă `Motocicleta_cross`, care moștenește `Motocicleta` și conține: **(1p)**
  - proprietățile: `nr_telescoape` (public get, protected set)
  - un constructor pentru inițializarea membrilor
  - o metodă `Salut()` care să afișeze `'Salut, sunt Motocicleta_cross'`
  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor
- O clasă `Motocicleta_speed`, care moștenește `Motocicleta` și conține: **(1p)**
  - proprietățile: `viteza_maxima` (public get, protected set)
  - un constructor pentru inițializarea membrilor
  - o metodă `Salut()` care să afișeze `'Salut, sunt Motocicleta_speed'`
  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Motocicleta` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 02

1. Implementați o funcție `Romeo30()` care primește 2 numere `ushort` și le inversează dacă ambele sunt pare. **(2p)**
2. Implementați o funcție `Hotel30()` care primește o listă de `int` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `int` și returnează un `boolean`, iar a doua funcție primește un `int` și returnează un `int`.

`Hotel30()` va returna o nouă listă în care toate elementele din lista inițială care îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Bere`, care conține: **(1p)**

  - proprietățile: `marca` `pret` `procent_alcool` (public get, protected set)

  - un constructor pentru inițializarea proprietăților

  - o metodă abstractă `void Salut()`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Bere_neagra`, care moștenește `Bere` și conține: **(1p)**

  - proprietățile: `procent_caramel` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Bere_neagra'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Bere_neflitrata`, care moștenește `Bere` și conține: **(1p)**

  - proprietățile: `procent_suspensii` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Bere_neflitrata'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Bere` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 03

1. Implementați o funcție `Delta23()` care primește 2 numere `int` și le inversează dacă ambele sunt mai mari decât 57. **(2p)**
2. Implementați o funcție `Charlie22()` care primește o listă de `uint` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `uint` și returnează un `boolean`, iar a doua funcție primește un `uint` și returnează un `uint`.

`Charlie22()` va returna o nouă listă în care toate elementele din lista inițială care îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Masina`, care conține: **(1p)**

  - proprietățile: `model` `marca` `capacitate_cilindrica` `an_fabricatie` (public get, protected set)

  - un constructor pentru inițializarea proprietăților

  - o metodă abstractă `void Salut()`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Masina_de_curse`, care moștenește `Masina` și conține: **(1p)**

  - proprietățile: `timp_accelerare_0_100` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Masina_de_curse'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Masina_de_gunoi`, care moștenește `Masina` și conține: **(1p)**

  - proprietățile: `tonaj` (public get, protected set)

  - un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Masina_de_gunoi'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Masina` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 04

1. Implementați o funcție `Romeo11()` care primește 2 numere `double` și le inversează dacă ambele sunt mai mari decât 17. **(2p)**
2. Implementați o funcție `Bravo33()` care primește o listă de `int` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `int` și returnează un `boolean`, iar a doua funcție primește un `int` și returnează un `int`.

`Bravo33()` va returna o nouă listă în care toate elementele din lista inițială care îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Vin`, care conține: **(1p)**

  - proprietățile: `marca` `pret` `an` `procent_alcool` (public get, protected set)

  - un constructor pentru inițializarea proprietăților

  - o metodă abstractă `void Salut()`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Vin_rosu`, care moștenește `Vin` și conține: **(1p)**

  - proprietățile: `procent_aroma_butuc` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Vin_rosu'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Vin_alb`, care moștenește `Vin` și conține: **(1p)**

  - proprietățile: `procent_sulfiti` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Vin_alb'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Vin` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 05

1. Implementați o funcție `Mike33()` care primește 2 numere `float` și le inversează dacă ambele sunt mai mari decât 58. **(2p)**
2. Implementați o funcție `Bravo40()` care primește o listă de `short` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `short` și returnează un `boolean`, iar a doua funcție primește un `short` și returnează un `short`.

`Bravo40()` va returna o nouă listă în care toate elementele din lista inițială care îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Student`, care conține: **(1p)**

- proprietățile: `nume` `prenume` `varsta` `email` (public get, protected set)

- un constructor pentru inițializarea proprietăților

- o metodă abstractă `void Salut()`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Student_bursier`, care moștenește `Student` și conține: **(1p)**

- proprietățile: `bursa` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Student_bursier'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Student_sportiv`, care moștenește `Student` și conține: **(1p)**

- proprietățile: `nr_medalii` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Student_sportiv'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Student` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 06

1. Implementați o funcție `X_Ray04()` care primește 2 numere `float` și le inversează dacă ambele sunt mai mari decât 62. **(2p)**
2. Implementați o funcție `Charlie01()` care primește o listă de `long` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `long` și returnează un `boolean`, iar a doua funcție primește un `long` și returnează un `long`.

`Charlie01()` va returna o nouă listă în care toate elementele din lista inițială care îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Motocicleta`, care conține: **(1p)**

- proprietățile: `marca` `model` `viteza_maxima` (public get, protected set)

- un constructor pentru inițializarea proprietăților

- o metodă abstractă `void Salut()`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Motocicleta_cross`, care moștenește `Motocicleta` și conține: **(1p)**

- proprietățile: `nr_telescoape` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Motocicleta_cross'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Motocicleta_speed`, care moștenește `Motocicleta` și conține: **(1p)**

  - proprietățile: `viteza_maxima` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Motocicleta_speed'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Motocicleta` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 07

1. Implementați o funcție `Golf34()` care primește 2 numere `ulong` și le inversează dacă ambele sunt pare. **(2p)**
2. Implementați o funcție `Mike21()` care primește o listă de `short` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `short` și returnează un `boolean`, iar a doua funcție primește un `short` și returnează un `short`.

`Mike21()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Tara`, care conține: **(1p)**

  - proprietățile: `nume` `nume_continent` `populatie` (public get, protected set)

  - un constructor pentru inițializarea proprietăților

  - o metodă abstractă `void Salut()`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Tara_petroliera`, care moștenește `Tara` și conține: **(1p)**

  - proprietățile: `venit_pe_locuitor` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Tara_petroliera'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Tara_turistica`, care moștenește `Tara` și conține: **(1p)**

  - proprietățile: `nr_turisti_anuali` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Tara_turistica'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Tara` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 08

1. Implementați o funcție `Papa22()` care primește 2 numere `double` și le inversează dacă ambele sunt mai mari decât 89. **(2p)**
2. Implementați o funcție `Echo34()` care primește o listă de `float` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `float` și returnează un `boolean`, iar a doua funcție primește un `float` și returnează un `float`.

`Echo34()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Motocicleta`, care conține: **(1p)**

  - proprietățile: `marca` `model` `viteza_maxima` (public get, protected set)

  - un constructor pentru inițializarea proprietăților

  - o metodă abstractă `void Salut()`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Motocicleta_cross`, care moștenește `Motocicleta` și conține: **(1p)**

  - proprietățile: `nr_telescoape` (public get, protected set)

  - un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Motocicleta_cross'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Motocicleta_speed`, care moștenește `Motocicleta` și conține: **(1p)**

- proprietățile: `viteza_maxima` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Motocicleta_speed'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Motocicleta` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 09

1. Implementați o funcție `India03()` care primește 2 numere `long` și le inversează dacă ambele sunt pare. **(2p)**
2. Implementați o funcție `Bravo42()` care primește o listă de `short` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `short` și returnează un `boolean`, iar a doua funcție primește un `short` și returnează un `short`.

`Bravo42()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Judet`, care conține: **(1p)**

- proprietățile: `nume` `nume_tara` `populatie` `suprafata` (public get, protected set)

- un constructor pentru inițializarea proprietăților

- o metodă abstractă `void Salut()`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Judet_normal`, care moștenește `Judet` și conține: **(1p)**

- proprietățile: `nr_intreprinderi` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Judet_normal'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Judet_agricol`, care moștenește `Judet` și conține: **(1p)**

- proprietățile: `suprafata_cultivata` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Judet_agricol'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Judet` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 10

1. Implementați o funcție `Quebec41()` care primește 2 numere `byte` și le inversează dacă ambele sunt impare. **(2p)**
2. Implementați o funcție `Uniform10()` care primește o listă de `byte` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `byte` și returnează un `boolean`, iar a doua funcție primește un `byte` și returnează un `byte`.

`Uniform10()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Bere`, care conține: **(1p)**

- proprietățile: `marca` `pret` `procent_alcool` (public get, protected set)

- un constructor pentru inițializarea proprietăților

- o metodă abstractă `void Salut()`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Bere_neagra`, care moștenește `Bere` și conține: **(1p)**

- proprietățile: `procent_caramel` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Bere_neagra'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Bere_neflitrata`, care moștenește `Bere` și conține: **(1p)**

- proprietățile: `procent_suspensii` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Bere_neflitrata'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Bere` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 11

1. Implementați o funcție `Juliet14()` care primește 2 numere `int` și le inversează dacă ambele sunt impare. **(2p)**
2. Implementați o funcție `Tango43()` care primește o listă de `byte` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `byte` și returnează un `boolean`, iar a doua funcție primește un `byte` și returnează un `byte`.

`Tango43()` va returna o nouă listă în care toate elementele din lista inițială care îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Bicicleta`, care conține: **(1p)**

- proprietățile: `marca` `pret` `an` (public get, protected set)

- un constructor pentru inițializarea proprietăților

- o metodă abstractă `void Salut()`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Bicicleta_munte`, care moștenește `Bicicleta` și conține: **(1p)**

  - proprietățile: `latime_ghidon` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Bicicleta_munte'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Bicicleta_oras`, care moștenește `Bicicleta` și conține: **(1p)**

  - proprietățile: `nr_viteze` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Bicicleta_oras'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Bicicleta` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 12

1. Implementați o funcție `Hotel01()` care primește 2 numere `ulong` și le inversează dacă ambele sunt mai mari decât 89. **(2p)**
2. Implementați o funcție `Victor23()` care primește o listă de `byte` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `byte` și returnează un `boolean`, iar a doua funcție primește un `byte` și returnează un `byte`.

`Victor23()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Oras`, care conține: **(1p)**

  - proprietățile: `nume` `nume_judet` `nr_locuitori` `suprafata` (public get, protected set)

  - un constructor pentru inițializarea proprietăților

  - o metodă abstractă `void Salut()`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Oras_resedinta`, care moștenește `Oras` și conține: **(1p)**

  - proprietățile: `nr_consilieri` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Oras_resedinta'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Oras_municipiu`, care moștenește `Oras` și conține: **(1p)**

  - proprietățile: `nr_autobuse` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Oras_municipiu'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Oras` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 13

1. Implementați o funcție `Victor24()` care primește 2 numere `double` și le inversează dacă ambele sunt mai mari decât 40. **(2p)**
2. Implementați o funcție `Sierra04()` care primește o listă de `ushort` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `ushort` și returnează un `boolean`, iar a doua funcție primește un `ushort` și returnează un `ushort`.

`Sierra04()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Vin`, care conține: **(1p)**

  - proprietățile: `marca` `pret` `an` `procent_alcool` (public get, protected set)

  - un constructor pentru inițializarea proprietăților

- o metodă abstractă `void Salut()`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Vin_rosu`, care moștenește `Vin` și conține: **(1p)**

- proprietățile: `procent_aroma_butuc` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Vin_rosu'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Vin_alb`, care moștenește `Vin` și conține: **(1p)**

- proprietățile: `procent_sulfiti` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Vin_alb'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Vin` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 14

1. Implementați o funcție `Zulu00()` care primește 2 numere `uint` și le inversează dacă ambele sunt mai mari decât 63. **(2p)**
2. Implementați o funcție `Whiskey33()` care primește o listă de `long` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `long` și returnează un `boolean`, iar a doua funcție primește un `long` și returnează un `long`.

`Whiskey33()` va returna o nouă listă în care toate elementele din lista inițială care îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Student`, care conține: **(1p)**

- proprietățile: `nume` `prenume` `varsta` `email` (public get, protected set)

- un constructor pentru inițializarea proprietăților

- o metodă abstractă `void Salut()`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Student_bursier`, care moștenește `Student` și conține: **(1p)**

- proprietățile: `bursa` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Student_bursier'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Student_sportiv`, care moștenește `Student` și conține: **(1p)**

- proprietățile: `nr_medalii` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Student_sportiv'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Student` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 15

1. Implementați o funcție `Golf24()` care primește 2 numere `short` și le inversează dacă ambele sunt impare. **(2p)**
2. Implementați o funcție `Yankee21()` care primește o listă de `ushort` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `ushort` și returnează un `boolean`, iar a doua funcție primește un `ushort` și returnează un `ushort`.

`Yankee21()` va returna o nouă listă în care toate elementele din lista inițială care îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Student`, care conține: **(1p)**

- proprietățile: `nume` `prenume` `varsta` `email` (public get, protected set)

- un constructor pentru inițializarea proprietăților

- o metodă abstractă `void Salut()`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Student_bursier`, care moștenește `Student` și conține: **(1p)**

- proprietățile: `bursa` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Student_bursier'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Student_sportiv`, care moștenește `Student` și conține: **(1p)**

- proprietățile: `nr_medalii` (public get, protected set)

- un constructor pentru inițializarea membrilor

- o metodă `Salut()` care să afișeze `'Salut, sunt Student_sportiv'`

- o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Student` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 16

1. Implementați o funcție `Foxtrot33()` care primește 2 numere `long` și le inversează dacă ambele sunt impare. **(2p)**
2. Implementați o funcție `Victor00()` care primește o listă de `ushort` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `ushort` și returnează un `boolean`, iar a doua funcție primește un `ushort` și returnează un `ushort`.

`Victor00()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Motocicleta`, care conține: **(1p)**
  - proprietățile: `marca` `model` `viteza_maxima` (public get, protected set)
  - un constructor pentru inițializarea proprietăților
  - o metodă abstractă `void Salut()`
  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor
- O clasă `Motocicleta_cross`, care moștenește `Motocicleta` și conține: **(1p)**
  - proprietățile: `nr_telescoape` (public get, protected set)
  - un constructor pentru inițializarea membrilor
  - o metodă `Salut()` care să afișeze `'Salut, sunt Motocicleta_cross'`
  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor
- O clasă `Motocicleta_speed`, care moștenește `Motocicleta` și conține: **(1p)**
  - proprietățile: `viteza_maxima` (public get, protected set)
  - un constructor pentru inițializarea membrilor
  - o metodă `Salut()` care să afișeze `'Salut, sunt Motocicleta_speed'`
  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor
- În programul principal, creați o colecție de tip `Motocicleta` și adăugați cel puțin 4 obiecte. **(1p)**
- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**
- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 17

1. Implementați o funcție `Quebec44()` care primește 2 numere `long` și le inversează dacă ambele sunt pare. **(2p)**
2. Implementați o funcție `Oscar23()` care primește o listă de `ulong` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `ulong` și returnează un `boolean`, iar a doua funcție primește un `ulong` și returnează un `ulong`.

`Oscar23()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Masina`, care conține: **(1p)**

  - proprietățile: `model` `marca` `capacitate_cilindrica` `an_fabricatie` (public get, protected set)

  - un constructor pentru inițializarea proprietăților

  - o metodă abstractă `void Salut()`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Masina_de_curse`, care moștenește `Masina` și conține: **(1p)**

  - proprietățile: `timp_accelerare_0_100` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Masina_de_curse'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Masina_de_gunoi`, care moștenește `Masina` și conține: **(1p)**

  - proprietățile: `tonaj` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Masina_de_gunoi'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Masina` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**

Student 18

1. Implementați o funcție `Uniform32()` care primește 2 numere `double` și le inversează dacă ambele sunt impare. **(2p)**
2. Implementați o funcție `Sierra13()` care primește o listă de `int` și două funcții.

Prima funcție reprezintă o condiție, adică primește un `int` și returnează un `boolean`, iar a doua funcție primește un `int` și returnează un `int`.

`Sierra13()` va returna o nouă listă în care toate elementele din lista inițială care **NU** îndeplinesc condiția primită ca parametru vor fi transformate în altă valoare de a doua funcție primită ca parametru. **(2p)**

3. Creați următoarea ierarhie de clase:

- O clasă abstractă `Masina`, care conține: **(1p)**

  - proprietățile: `model` `marca` `capacitate_cilindrica` `an_fabricatie` (public get, protected set)

  - un constructor pentru inițializarea proprietăților

  - o metodă abstractă `void Salut()`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Masina_de_curse`, care moștenește `Masina` și conține: **(1p)**

  - proprietățile: `timp_accelerare_0_100` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Masina_de_curse'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- O clasă `Masina_de_gunoi`, care moștenește `Masina` și conține: **(1p)**

  - proprietățile: `tonaj` (public get, protected set)

  - un constructor pentru inițializarea membrilor

  - o metodă `Salut()` care să afișeze `'Salut, sunt Masina_de_gunoi'`

  - o metodă `ToString()` care să returneze un string valorile concatenate ale membrilor

- În programul principal, creați o colecție de tip `Masina` și adăugați cel puțin 4 obiecte. **(1p)**

- Parcurgeți colecția și apelați metodele `Salut()`. **(0.5p)**

- Parcurgeți colecția și afișați reprezentările `ToString()` ale obiectelor. **(0.5p)**